数组传递：const 指针与其他方式的对比

文字提取与解析

图片文字为：**C. 用const指针传递数组，保证函数内不修改原数组**

知识点解析 ✅

在C++中，用 `const` 指针传递数组是一种**安全的编程实践**，核心作用如下：

1. **语法形式**

   Code block

   ```
   // 函数声明示例
   void printArray(const int* arr, int size);
   ```

   - `const int* arr` 表示：指针 `arr` 指向的是**常量整数**，函数内部**不能通过 `arr` 修改原数组的元素值**。
   - 数组传递时会退化为指针，`const` 修饰可以防止意外修改。
2. **核心优势**
   - **数据安全**：明确约束函数行为，避免函数误改原数组，符合“最小权限原则”。
   - **接口清晰**：调用者一眼可知该函数是“只读”操作，不会破坏传入的数据。
   - **兼容高效**：既保留了指针传递的高效性，又获得了值传递的安全性。
3. **注意区分**
   - `const int* arr`：指向常量的指针 → **不能修改指向的内容**，可以修改指针指向。
   - `int* const arr`：常量指针 → **不能修改指针指向**，可以修改指向的内容。
   - `const int* const arr`：双重const → 指针和指向内容都不能修改，最严格的只读约束。

示例代码

Code block

```
#include <iostream>
```

```
using namespace std;

// 只读打印函数，const保证不会修改原数组
void printArray(const int* arr, int size) {
    for (int i = 0; i < size; ++i) {
        cout << arr[i] << " ";
        // arr[i] = 100; // 编译报错！const修饰下无法修改元素
    }
    cout << endl;
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    printArray(arr, 5); // 安全传递，原数组不会被修改
    return 0;
}
```

总结 💡

- **适用场景**：函数只需要读取数组数据，不需要修改时，必须用 `const` 指针传递。
- **最佳实践**：养成“能const就const”的习惯，让编译器帮你检查代码，避免逻辑错误。

你要我帮你对比一下**值传递、普通指针传递、const指针传递**这三种数组传递方式的区别吗？

（注：文档部分内容可能由 AI 生成）